

Nama : Arevanda Lazuardi Arrayan

NIM : 245150407111091

Tugas Modul 3 Event, State, dan Implisit Intent pada Jetpack Compose

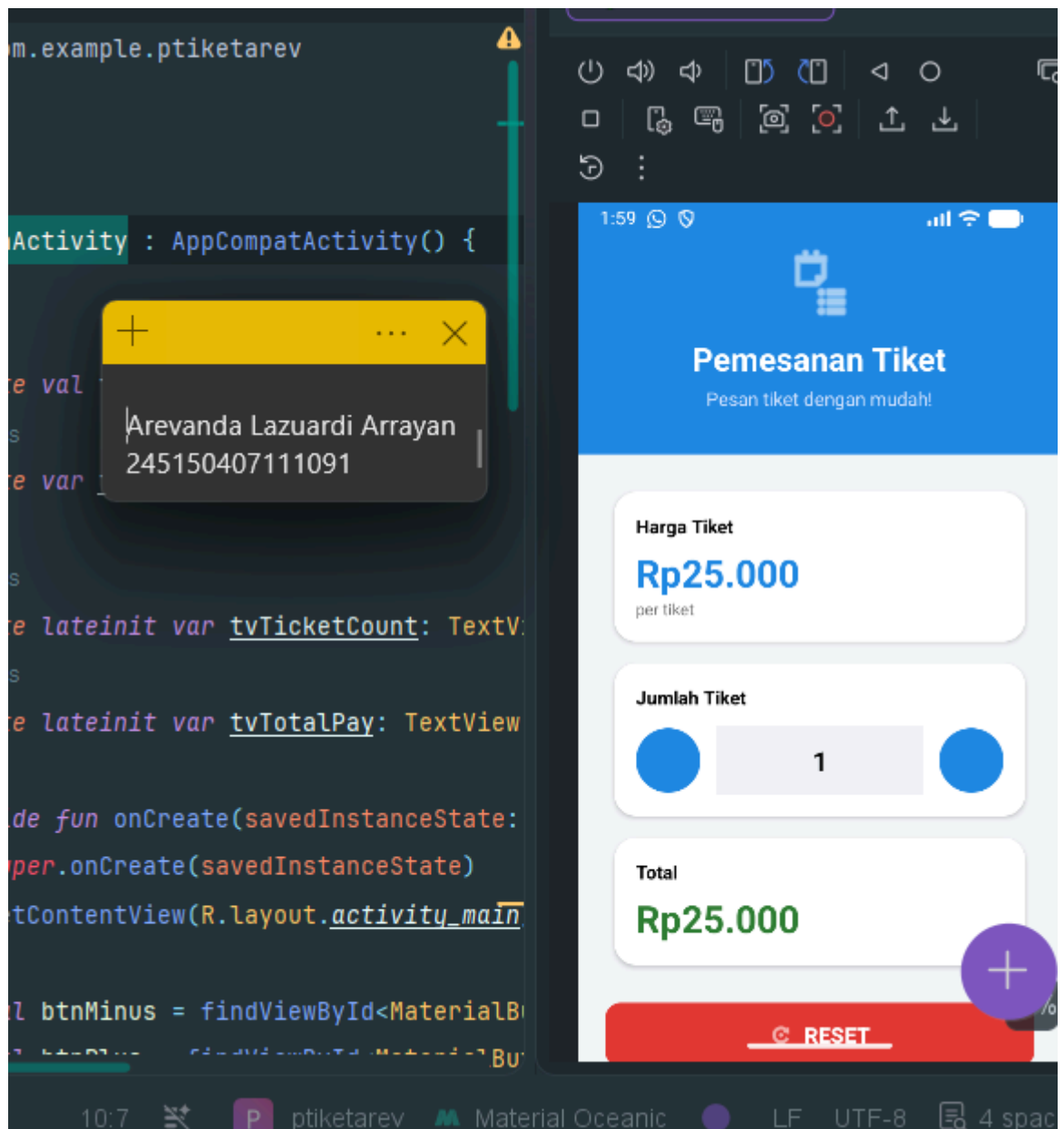
Aktivitas Percobaan Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

1. Kedua aplikasi dijalankan dan screenshot diambil sebelum dan sesudah tombol diklik. Sebelum tombol diklik teks menampilkan "Belum diklik", setelah tombol "Ubah Teks" diklik teks berubah menjadi "Tombol sudah diklik". Hasilnya sama pada project XML/View maupun Compose.
2. Objek yang diubah secara langsung pada project XML/View adalah TextView (tvMessage). Baris kodenya: `tvMessage.text = "Tombol sudah diklik"`
3. Nilai yang diubah pada project Compose adalah state message. Baris kodenya: `message = "Tombol sudah diklik"`
4. Pada Compose tidak ada perintah seperti `tvMessage.text` atau `setText()` karena Compose menggunakan pendekatan deklaratif. Developer tidak mengakses objek Text untuk mengubah propertinya secara langsung. `Text(text = message)` hanya mendeklarasikan bahwa teks yang tampil berasal dari state message, sehingga cukup state-nya yang diubah dan Compose yang memperbarui tampilan.
5. Alur perubahan pada XML/View: event (tombol diklik melalui `setOnClickListener`) -> View (program mengambil referensi tvMessage dan mengubah properti text-nya secara langsung) -> perubahan tampilan (teks pada layar berubah).
6. Alur perubahan pada Compose: event (`onClick` pada Button dijalankan) -> state (nilai message diubah menjadi "Tombol sudah diklik") -> recomposition (Compose menjalankan ulang composable yang membaca message) -> perubahan tampilan (Text menampilkan nilai message yang baru).
7. Jika message diganti menjadi variabel biasa (`var message = "Belum diklik"`), teks pada layar tidak akan berubah ketika tombol diklik. Penyebabnya, variabel biasa tidak diamati oleh Compose sehingga perubahan nilainya tidak memicu recomposition. Selain itu, jika recomposition terjadi karena sebab lain, nilainya akan kembali ke "Belum diklik" karena tidak disimpan oleh remember.
8. Kesimpulan: Cara berpikir imperatif adalah memerintahkan langkah demi langkah, yaitu mengambil komponen UI lalu mengubah propertinya secara manual. Cara berpikir deklaratif adalah mendefinisikan tampilan berdasarkan state saat ini, sehingga ketika state berubah, framework yang memperbarui UI secara otomatis.

Dengan pendekatan deklaratif, developer cukup fokus mengelola state, dan tampilan selalu mengikuti state tersebut.

Aktivitas Percobaan Event Handling State

1. Hasil Screenshoot

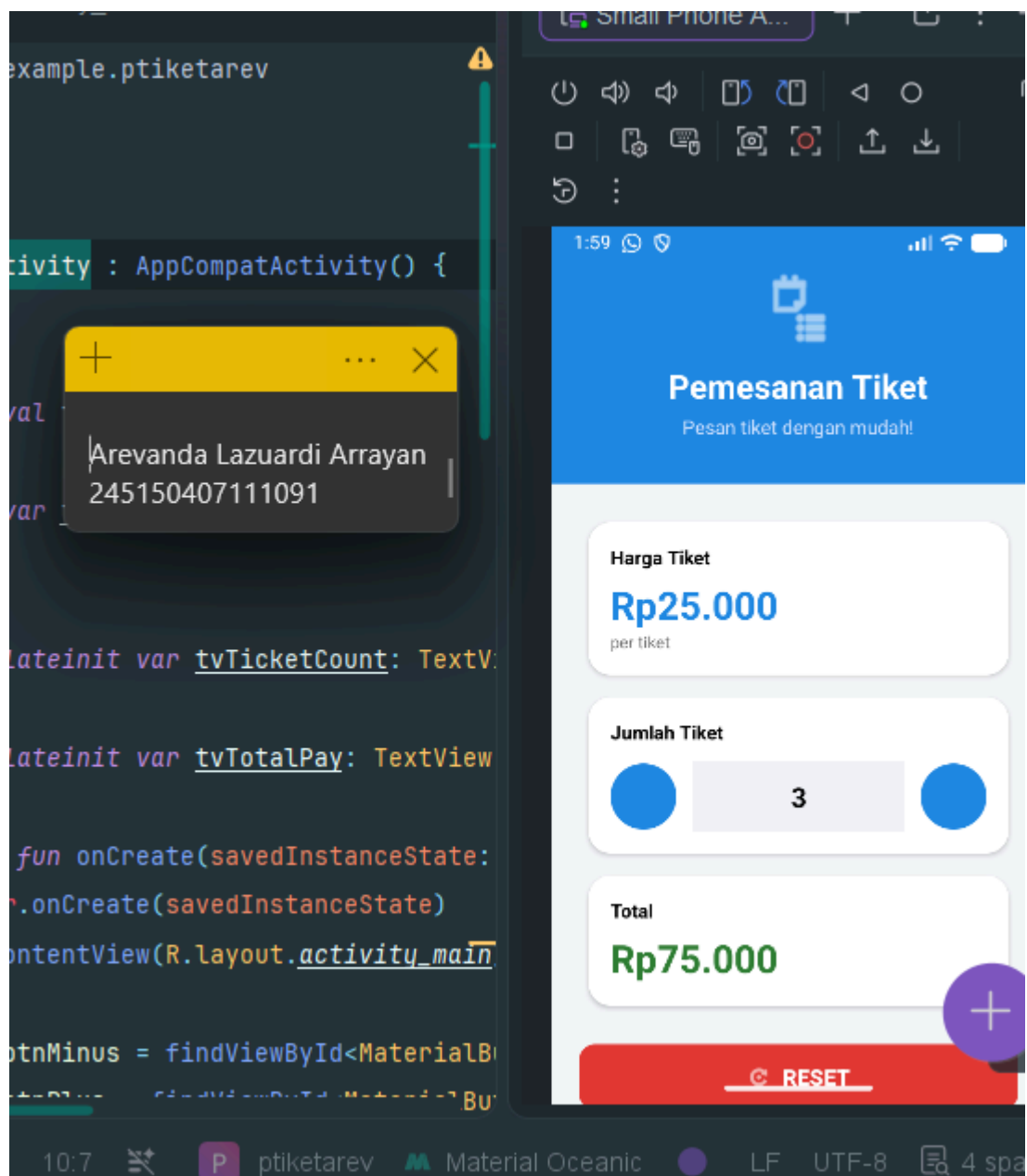


2. LogCat menampilkan "Counter berubah : 1", "Counter berubah : 2", "Counter berubah : 3", dan seterusnya, bertambah setiap kali tombol diklik.
3. Tampilan tidak berubah, tetap "Counter : 0"

4. Variabel count adalah variabel biasa, bukan state. Nilainya memang bertambah (terlihat di LogCat), tetapi Compose tidak mengetahui perubahan tersebut sehingga tidak terjadi recomposition dan Text tidak diperbarui
5. Langkah selanjutnya adalah mengubah count menjadi state pada composable EventStateTest
6. Kode "var count = 0" diubah menjadi "var count by remember { mutableStateOf(0)"
7. Kode lengkap EventStateTest disesuaikan dengan kode yang ada pada modul
8. Program dijalankan kembali, LogCat difilter dengan "TEST", lalu tombol "Tambah" diklik beberapa kali. Screenshot: tampilan menunjukkan angka sesuai jumlah klik, misalnya "Counter : 5"
9. LogCat tetap menampilkan "Counter berubah : 1", "Counter berubah : 2", "Counter berubah : 3", dan seterusnya
10. Angka pada Text bertambah satu setiap kali tombol "Tambah" diklik
11. Variabel count sekarang berupa state (mutableStateOf) yang diamati oleh Compose. Ketika nilainya berubah, Compose menjalankan recomposition sehingga Text menampilkan nilai terbaru. Remember menjaga nilai state agar tidak kembali ke 0 setiap terjadi recomposition. Pada percobaan sebelumnya tidak ada state yang diamati, sehingga UI tidak pernah diperbarui
12. Recomposition adalah proses ketika Compose menjalankan ulang composable yang membaca state yang berubah, supaya UI sesuai dengan state terbaru. Hanya bagian UI yang bergantung pada state tersebut yang digambar ulang
13. State baru bernama text (String) ditambahkan, dan OutlinedTextField dipanggil di dalam Column di bawah Button "Tambah"
14. Program dijalankan kembali, LogCat difilter dengan "TEST". Screenshot: tampil counter, tombol "Tambah", dan OutlinedTextField di bawahnya
15. Setiap huruf yang diketik memunculkan satu log berisi isi teks terbaru, misalnya "Text berubah : a", "Text berubah : ab", "Text berubah : abc"
16. Huruf yang diketik muncul pada Text Field
17. Setiap ketikan memanggil onChange, yang menjalankan "text = it" sehingga state text berubah. Perubahan tersebut memicu recomposition dan nilai (value) pada Text Field diperbarui. Tanpa pembaruan state ini, huruf yang diketik tidak akan tampil
18. State pada program terakhir: count (Int) dan text (String)
19. Event pada program terakhir: onClick pada Button "Tambah" dan onChange pada OutlinedTextField
20. onClick dijalankan setiap kali pengguna menekan tombol "Tambah". onChange dijalankan setiap kali isi Text Field berubah, baik saat mengetik, menghapus, maupun menempel teks

Aktivitas Percobaan Implisit Intent

1. Program berhasil dijalankan pada emulator atau perangkat Android fisik, kemudian setiap tombol diuji.
2. Klik Buka Browser: browser terbuka dan menampilkan <https://www.google.com>.
3. Klik Buka Telepon: aplikasi dialer terbuka dengan nomor 08123456789 sudah terisi, tetapi belum melakukan panggilan.
4. Klik Kirim Email: muncul chooser "Pilih aplikasi email". Setelah memilih aplikasi, form email terbuka dengan penerima test@gmail.com.
5. Klik Buka Maps: aplikasi peta terbuka pada koordinat -7.250445, 112.768845.
6. Hasil Screenshoot



7. Action yang digunakan pada kode adalah `Intent.ACTION_VIEW`, `Intent.ACTION_DIAL`, dan `Intent.ACTION_SENDTO`.
8. `Intent.ACTION_VIEW`: menampilkan data kepada pengguna melalui aplikasi yang sesuai (dipakai untuk membuka URL di browser dan lokasi di Maps).
9. `Intent.ACTION_DIAL`: membuka aplikasi telepon dengan nomor yang sudah terisi tanpa langsung menelepon.
10. `Intent.ACTION_SENDTO`: mengirim pesan ke tujuan yang ditentukan oleh data URI (dipakai untuk membuka aplikasi email).
11. Perbedaan action: `ACTION_VIEW` menampilkan atau membuka data seperti halaman web dan peta, `ACTION_DIAL` membuka dialer dengan nomor terisi, dan `ACTION_SENDTO` mengirim pesan ke alamat tujuan tertentu seperti email.
12. Data yang digunakan pada kode berupa URI dengan skema `mailto:`, `https:`, dan `geo:`.
13. `Uri.parse("mailto:test@gmail.com")`: menentukan alamat email tujuan sehingga aplikasi email terbuka dengan penerima sudah terisi.
14. `Uri.parse("https://www.google.com")`: menentukan alamat web yang dibuka sehingga browser menampilkan halaman tersebut.
15. `Uri.parse("geo:-7.250445,112.768845")`: menentukan koordinat lokasi sehingga aplikasi peta menampilkan lokasi tersebut.
16. Fungsi data pada intent: menentukan objek yang dikenai action sekaligus jenisnya melalui skema URI (`https:`, `tel:`, `mailto:`, `geo:`). Data menjawab pertanyaan "action dilakukan terhadap apa".
17. Hubungan action dan data: Android mencocokkan action dan data (skema URI) dari intent dengan intent-filter milik aplikasi-aplikasi yang terpasang. Hanya aplikasi yang filternya cocok dengan kombinasi keduanya yang muncul sebagai pilihan. Contohnya, `ACTION_VIEW` dengan data `https:` cocok dengan browser, sedangkan `ACTION_VIEW` dengan data `geo:` cocok dengan aplikasi peta. Jika lebih dari satu aplikasi cocok, pengguna diminta memilih melalui chooser.